

Variables and Statements

COSC 1210 (CS I) Fall 26 | Augustana U.

Table of Contents

- [1. Objectives](#)
- [2. Variables are really pointers to objects](#)
- [3. Python vs. CPython skip](#)
- [4. Variable names](#)
- [5. PRACTICE Variable names](#)
- [6. Expressions vs. statements](#)
- [7. print function revisited](#)
- [8. Code-along: Pythagoras' Theorem](#)
- [9. PRACTICE distance between two points](#)
- [10. Looking ahead: name the formula with a function](#)
- [11. Summary](#)
- [12. Glossary](#)

1. Objectives

Source: Downey, Think Python (3e), chapter 2.

By the end of this lesson, you can:

- use a **variable** as a name bound to an object, and know that assigning to it again **rebinds** the name to a new object
- follow the rules for legal variable names and avoid Python keywords
- tell an **expression** (evaluates to one value) from a **statement** (has an effect but no value), and = (assign) from == (test equality)
- say what print does with its arguments and why it hands back None
- combine text and numbers in output with an **f-string** or with `str()`
- import a module and reach its names with the dot, e.g. `math.pi`
- pass **arguments** to a function such as `print`, `len`, `type` or `round`
- turn a formula - the Pythagorean theorem - into Python with `**` and `** 0.5`

Assignments:

- Home: Pay Stub Calculator
- Bonus: Circle Calculator, Change Maker

2. Variables are really pointers to objects

- You can think of a variable as a storage container.
- Technically, it is a **reference** to a **memory** location.
- When you change the value of the variable, it simply refers to a new location in machine memory. This is also called **rebinding**.
- Example: Define a variable `spam` and make it point at the value 33

```
spam = 33
print(spam)
```

```
33
```

- Now point spam at another value:

```
spam = 3.14
print(spam)
```

```
3.14
```

- You can see that it looks as if spam had just sat there like a box, and you had taken 33 out and replaced it by 3.14. In truth, there is a single name spam that gets **repointed**: first to an **integer** object 33, then to a **float** object 3.14 - two different objects of two different types.

3. Python vs. CPython skip

- The exact sizes depend on which Python you run. *CPython* - the reference implementation, written in C, the one you download from python.org and the one Google Colab and Jupyter run - is almost certainly what you have. Other implementations (PyPy, Jython, MicroPython) store objects differently, so the numbers below are a fact about CPython, not a rule of the language.
- You can check how much memory each object uses with `sys.getsizeof`:

```
import sys
print(sys.getsizeof(33))    # the integer object
print(sys.getsizeof(3.14)) # the float object
```

```
28
24
```

- Those numbers include the object's bookkeeping - a type pointer and a reference count - not just the raw value. A float wraps an 8-byte IEEE-754 double; a Python int has no fixed width at all and grows as the number gets bigger. (In C, an int is typically just 4 bytes and a double 8 bytes with nothing else attached.)

4. Variable names

- You cannot use any of the reserved Python keywords. Here is a list:

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

- You don't need to memorize this list. Your editor will help with syntax highlighting, which is sensitive to keywords, and the compiler will get an aneurysm:

```
print(10)      # print is reserved
if = 0         # if is reserved
del(x)         # del is reserved
delete(x)      # delete is not reserved
var = 100      # var is not reserved
return = 40    # return is reserved
None = 50      # None is reserved
print("None")  # None in "None" is not reserved
```

- Otherwise, there are few rules:
 1. Variable names can be as long as you like (and need them to be)
 2. They can contain letters and numbers
 3. But they cannot begin with a number
 4. The only punctuation mark allowed is the underscore _

5. PRACTICE Variable names

Six names in the program below break one of the rules above. Rewrite it so that every name is legal, the program runs, and it still computes the same train fare. Keep each name's meaning obvious.

```
class = "first"          # ticket class
1st_leg_miles = 120
2nd_leg_miles = 85
price per mile = 0.19
total-miles = 1st_leg_miles + 2nd_leg_miles
fare$ = total-miles * price per mile
print(class, "class fare:", fare$)
```

When it is fixed, the output should be:

```
first class fare: 38.95
```

Solution skip

```
TicketClass = "first"  # CamelCase
first_leg_miles = 120
second_leg_miles = 85
price_per_mile = 0.19
total_miles = first_leg_miles + second_leg_miles
fare = total_miles * price_per_mile
print(TicketClass, "class fare:", fare)
```

```
first class fare: 38.95
```

Broken name	Rule broken	Fixed name
`class`	reserved keyword	`TicketClass`
`1st_leg_miles`	cannot begin with a number	`first_leg_miles`
`2nd_leg_miles`	cannot begin with a number	`second_leg_miles`

`price per mile`	no spaces allowed	`price_per_mile`	
`total-miles`	`.` not allowed, only `_`	`total_miles`	
`fare\$`	only letters, digits and `_`	`fare`	

6. Expressions vs. statements

- **Expressions** are always evaluated to a **single value** of a specific **type**.
- An arithmetic expression results in a number (int or float):

```
print(19 + 199*2 / 100 % 4)
```

22.98

- A boolean expression results in a truth (bool) value:

```
print(2 + 2 == 5) # check if `2 + 2` is equal to `5`
```

False

- A string expression results in a string (str):

```
print("The postman rang " + str(2) + " times.")
```

The postman rang 2 times.

- A **statement** is code that has an effect but **no value**.
- An import statement has the effect of making data and functions available - there is no output:

```
# math constants like math.pi and functions like math.pow
import math
```

- An **assignment** statement creates a variable and points it at a value but it does not have a value of its own:

```
age = 25
print(f"We've been happily married for {age} years.")
```

We've been happily married for 25 years.

- Notice that = is the assignment operator, but == is the operator that tests for equality.

7. print function revisited

- The print function displays the results of an expression:

```
print(2 + 2)
```

```
4
```

- You already know that it does not return anything as such, it only displays its arguments. That's the difference between a **vending machine** and a **window**.
- `len()` is a vending machine: You run it and it returns a value

```
print(len("hello there"))
```

```
11
```

- `print()` is a window: it only shows, it does not give. It gives nothing, in Python-speak that's `None`.

```
x = print(100) # prints 100 and assigns nothing to x
print(x)       # x has nothing - prints None
print(type(print(x))) # None is in a class of its own, NoneType
```

```
100
None
None
<class 'NoneType'>
```

- Or as a one-liner:

```
print(print("hello")) # first print prints, second print has nothing
# (`None`) to print
```

```
hello
None
```

- Three different ways to print the value of `pi`:

```
print('The value of pi is approximately', math.pi) # adds a space
print(f"The value of pi is approximately {math.pi}") # flexible format string
print("The value of pi is approximately " + str(math.pi)) # no space added
```

```
The value of pi is approximately 3.141592653589793
The value of pi is approximately 3.141592653589793
The value of pi is approximately 3.141592653589793
```

8. Code-along: Pythagoras' Theorem

Nothing new here - this is a short code-along that puts the arithmetic from chapter 1 and the variables and `print` from chapter 2 to work on one formula. It ends with a lab you solve in class.

The theorem: for a right triangle with legs a and b, the hypotenuse c satisfies

$$c^2 = a^2 + b^2 \rightarrow c = \sqrt{a^2 + b^2}$$

The formula $c^2 = a^2 + b^2$ translates straight to Python. Store the legs, then build the right-hand side.

```
a = 3
b = 4
c_squared = a**2 + b**2
print(c_squared)
```

25

To get c, undo the square - raise to the power 0.5, the same trick you used for $\sqrt{2}$.

```
c = c_squared ** 0.5
print(c)
```

5.0

The answer prints as 5.0, not 5: `** 0.5` always produces a float.

Now the whole thing on one line. The parentheses force `a**2 + b**2` first, then the outer power - ordinary order of operations. Format the result with an **f-string** to give the following output - but do not write the numbers 3,4,5.0 in there by hand:

A 3-4 right triangle has hypotenuse 5.0

```
c = (a**2 + b**2) ** 0.5
print(f"A {a}-{b} right triangle has hypotenuse {c}")
```

A 3-4 right triangle has hypotenuse 5.0

Check it: square the hypotenuse and see whether we get `c_squared` back. `==` tests equality (not `=`, which assigns) and the whole line is a boolean expression.

```
print(c**2 == c_squared)
```

True

9. PRACTICE distance between two points

- The distance between two points (x_1, y_1) and (x_2, y_2) on a graph is

$$\text{distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- This is the Pythagorean theorem, where the two legs are the horizontal and vertical gaps between the points.

Use the points **(1, 2)** and **(4, 6)**.

1. Store the four coordinates in four variables.
2. Compute the horizontal gap and the vertical gap, each in its own variable.
3. Compute the distance - first in two steps (gap squared, then the square root), then rewrite it as a single line.
4. Print the result in a sentence with an f-string.

Correct output:

```
Distance from (1, 2) to (4, 6) is 5.0
```

Sanity check: the gaps are 3 and 4, so this is the 3-4-5 triangle again.

Stretch: run it again for **(-2, -1)** and **(1, 3)**. Do the negative coordinates break anything? Why not?

Constraints - use only what we have already used:

- variables, =, arithmetic operators, **, ** 0.5, str, f-strings, print. No if, no loops, no imports.

Solution skip

```
# Store the four coordinates in four variables.
x1 = 1; y1 = 2
x2 = 4; y2 = 6
# Compute the horizontal gap and the vertical gap, each in its own variable.
dx = x2 - x1          # horizontal gap
dy = y2 - y1          # vertical gap
# Compute the distance in two steps: gap squared, square root
gap_squared = dx**2 + dy**2
distance = gap_squared ** 0.5
# distance written as a single line
distance = ((x2 - x1)**2 + (y2 - y1)**2) ** 0.5 # same thing, one line
print(f"Distance from ({x1}, {y1}) to ({x2}, {y2}) is {distance}")
```

```
Distance from (1, 2) to (4, 6) is 5.0
```

Stretch: with **(-2, -1)** and **(1, 3)** the gaps are $1 - (-2) = 3$ and $3 - (-1) = 4$, so the distance is 5.0 again. Squaring a negative gap gives a positive number, so the sign never reaches the answer.

10. Looking ahead: name the formula with a function

By now you have written $(a**2 + b**2) ** 0.5$ three or four times. A **function** lets you write it once, give it a name, and reuse it.

```
def hypotenuse(a, b):
    c = (a**2 + b**2) ** 0.5
    print(c)

hypotenuse(3, 4)
```

```
hypotenuse(5, 12)
hypotenuse(8, 15)
```

```
5.0
13.0
17.0
```

- `def` starts the definition; `a` and `b` are the **parameters**; the indented lines are the **body**. Calling `hypotenuse(3, 4)` runs the body with `a` set to 3 and `b` set to 4.
- This function **prints** its result, like every function we write for now. It does not hand the value back, so there is nothing to store:

```
x = hypotenuse(3, 4)    # the function prints 5.0...
print(x)               # ...but gives nothing back, so x is None
```

```
5.0
None
```

- Same story as `print` itself: it **shows**, it does not **give**. Handing a value back so you can keep computing with it needs `return`, which is chapter 6.

Writing your own functions is chapter 3.

11. Summary

- A **variable** is a name bound to an object. Assigning again does not move a value in and out of a box; it **rebinds** the name to a different object (rebinding).
- **Name rules**: letters, digits and the underscore only; no leading digit; no Python keyword.
- An **expression** evaluates to a single value of a type: arithmetic gives a number (`int`~/~`float`), a comparison gives a `bool`, string operations give a `str`.
- A **statement** has an effect but no value: an **assignment** (`=`) makes a variable, an `import` makes a module's names available.
- `=` assigns; `==` tests for equality.
- `print` **displays** its arguments and returns `None` - it shows, it does not give. `len`, `type`, `str` and the like **return** a value you can use.
- Three ways to build output: comma-separated arguments to `print` (adds a space), an **f-string**, or `+` with `str()`.
- Pythagoras in Python: `c = (a**2 + b**2) ** 0.5 ** 0.5` is a square root and always produces a `float`; the parentheses fix the order of operations.
- A **function** (`def`) names a formula once so you can call it again and again - the topic of chapter 3. For now our functions **print** their result; handing a value back with `return` is chapter 6.

12. Glossary

Term	Meaning
variable	A name that refers to an object
object	A value in memory, with a type (<code>int</code> , <code>float</code> , <code>str</code> , ...)

Term	Meaning
rebinding	Pointing an existing variable name at a different object
keyword	A reserved word (<code>if</code> , <code>import</code> , <code>class</code> , ...); cannot be a name
expression	Code that evaluates to a single value, e.g. <code>a**2 + b**2</code>
statement	Code that has an effect but no value (<code>assignment</code> , <code>import</code>)
evaluate	Work out the value of an expression
<code>=</code>	Assignment operator: bind a name to a value
<code>==</code>	Equality operator: test whether two values are equal
<code>int</code> / <code>float</code>	Whole number / number with a decimal point
<code>bool</code>	True or False; the result of a comparison
<code>str</code>	Text; also the function that converts a value to text
<code>None</code> / <code>NoneType</code>	"no value"; what <code>print</code> returns
<code>import</code> statement	Makes the names in a module available
module	A file of ready-made variables and functions (e.g. <code>math</code>)
<code>print</code>	Displays its arguments; returns <code>None</code>
<code>len</code>	Returns the number of characters in a string
<code>type</code>	Returns the type of a value
f-string	<code>f"...{expr}..."</code> - a string with values slotted in
<code>**</code>	Exponentiation; <code>x ** 0.5</code> is the square root of <code>x</code>
argument	A value passed to a function inside its parentheses
<code>def</code>	Starts a function definition (chapter 3)
<code>return</code>	Hands a value back from a function to its caller (chapter 6)

Author: Marcus Birkenkrahe

Created: 2026-09-07 Mon 20:46